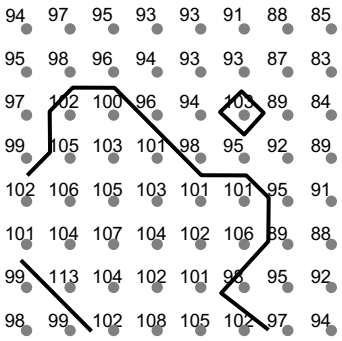
კვადრატების მარშის მეთოდი

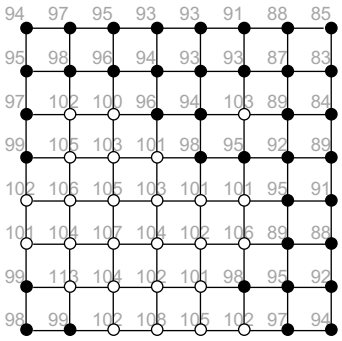


მოცემულია ტერიტორიის რუკა, რომელზეც დატანილია ცალკეული წერტილების გაზომილი სიმაღლეები. ამოცანა: ამ რუკაზე დავიტანოთ 100 მეტრის სიმაღლის შესაბამისი ჰორიზონტალური იზოხაზი.

ადამიანისთვის ამ ამოცანის ამოხსნა მონაცემების მცირე რაოდენობის შემთხვევაში არ არის რთული - უნდა უბრალოდ გავიყვანოთ ხაზი იქ, სადაც 100 მეტრზე დაბალი წერტილები, ესაზღვრება 100 მეტრზე მაღალ წერტილებს. (სურ.)

კვადრატების მარშის მეთოდიც მუშაობს მსგავსი მარტივი ლოგიკით

თავიდან მოცემულ წერტილებზე ავაგოთ ბადე. ბადის უჯრედების წვეროები (კუთხეები) არიან წერტილები აზომილი სიმაღლით. ხოლო ბადის უჯრედები - კვადრატები - სწორად ის „marching squares“ გახლავთ.



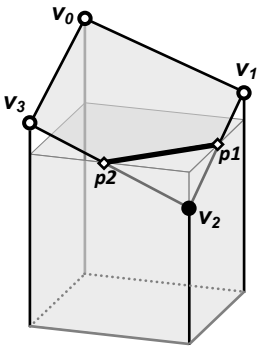
კვადრატის კუთხეები უნდა შევადაროთ მოცემულ სიმაღლეს, ანუ იმ სიმაღლეს, რომლის იზოხაზი უნდა გავავლოთ. ის წვეროები, რაც მოცემულ სიმაღლეზე მაღლაა, პირობითად დავარქვათ დადებითები წვეროები (H_i - $L > 0$, სადაც H_i - i-ური წვეროს სიმაღლეა, ხოლო L - მოცემული სიმაღლე.)

დადებითი წერტილები სქემაზე აღვნიშნოთ თეთრად, ხოლო უარყოფითი - შავად.

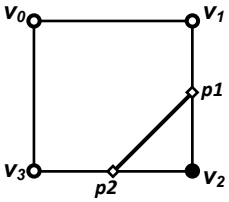
ბუნებრივია, რომ იმ კვადრატებში, სადაც ყველა წერტილი ერთი ფერისაა, ხაზი არ გაივლის, რათგან ხაზი - დადებითი და უარყოფითი წერტილების გამყოფია.

განვიხილოთ სხვა შემთხვევები, ანუ კვადრატები, რომლებშიც როგორც უარყოფითი, ასევე დადებითი წერტილებია. გავნსაზღვროთ, როგორ გაივლის ხაზი მოცემულ კვადრატებში. ბუნებრივია, რომ ხაზი გადაკვეთავს იმ მხარეებს (წიბოებს), რომელთა ბოლოები სხვადასხვა ფერისაა. თავიდან სიმარტივისთვის ჩავთვალოთ, რომ ხაზი ყოველთვის კვეთს კვადრატის მხარეების შუა წერტილებს. დავარქვათ ამ მეთოდს **შუა წერტილების მეთოდი.**

უფრო ზუსტ მეთოდს განვიხილავთ ქმემოთ.

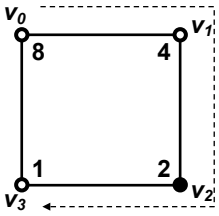


თუ სამი წერტილი ერთი ნიშნისაა, ხოლო მეოთხე - სხვა ნიშნის, მაშინ ხაზი გაივლის ამ მეოთხე წერტილის მიმდებარე წიბოებზე.



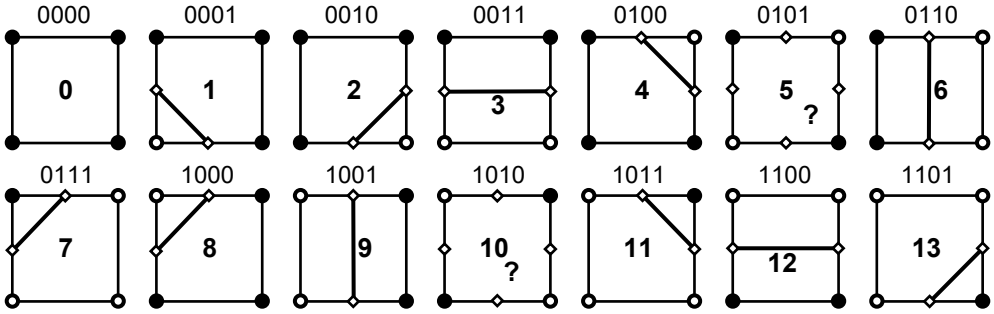
ილუსტრაციაზე ნაჩვენებია ამოცანის ფიზიკური ანალოგია: სამი წერტილი *(v0, v1, v3)* წყლის დონეზე მაღლაა, ხოლო წერტილი *v2* წყლის დონეზე დაბლაა. ხაზი *(p1 p2)* - პირობითი სანაპირო ზოლია.

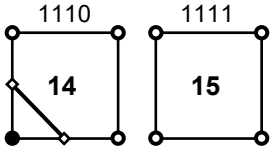
წვეროების სხვადასხვა მნიშვნელობებისთვის სულ შესაძლებელია (4 წვერო, თითოზე 2 ვარიანტი) $2^4 = 16$ კომბინაცია.



ნებისმიერი კომბინაციის აღსაწერად გამოვიყენოთ მარტივი მეთოდი: დავნომროთ წვეროები ზედა მარცხენა კუთხიდან საათის ისრის მიხედვით. თითო წვერო იქნება ოთხნიშნა ორობითი რიცხვის ერთი პოზიცია (ბიტი). თუ წერტილი შავია, მას ენიჭება ნოლი, თუ თეთრია - ორიანის შესაბამისი ხარისხი. ასე, სურათზე მოცემული მაგალითისთვის მივიღებთ 8+4+0+1 = 13.

ანუ შავი და თეთრი წერტილების ურთიერთგანლაგების ყველა ვარიანტისთვის შემუშავდა კოდი, რომლის მიხედვით შეიძლება ამ ვარიანტების სწრაფი აღწერა.





კვადრატში წვეროების განლაგების 16 ვარიანტი - თავისი ორობითი აღწერით და შესაბამისი რიცხვითი მნიშვნელობით/კოდით. ვარიანტებს 0 და 15 ხაზი არ აქვთ (ყველა წვერო ერთი ფერისაა). ვარიანტეში 5 და 10, კვადრატს და ხაზს ოთხ-ოთხი გადამკვეთი წერტილი აქვთ და ხაზების გავლა უფრო რთულია. ამ ოთხი წერტილის პრობლემას განვიხილავთ შემდგომში.

კვადრატის, ანუ მისი ოთხი წვეროს განლაგების კოდის `code` გამოთვლა.

კვადრატის ოთხი წვეროს მნიშვნელობები: `v0`, `v1`, `v2` და `v3`. და „წყლის“ სიმაღლეა `lev`.

შესაძლებელია მექანიკურად შევაჯამოთ ოთხივე წვეროს კოეფიციენტი:

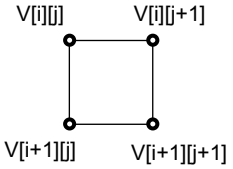
```
code = 0;
if(v0>lev) code +=8;
if(v1>lev) code +=4;
if(v2>lev) code +=2;
if(v3>lev) code++;
```

თუმცა, უკეთესია უფრო კომპაქტური და ეფექტური კოდის გამოყენება:

```
code = (v0>lev)<<3|(v1>lev)<<2|(v2>lev)<<1|(v3>lev);
```

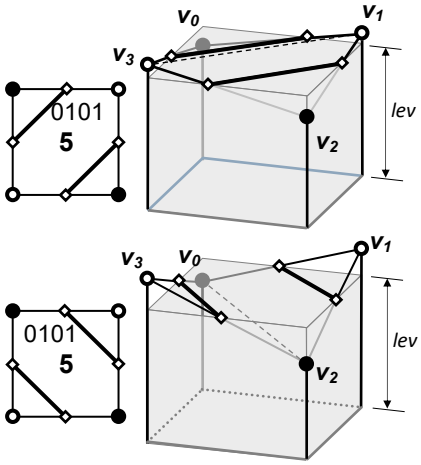
ალგორითმის მსვლელობა შეგვიძლია შემდეგნაირად წარმოვადგინოთ:

N– წვეროების მნიშვნელობების მატრიცის ზომა (NxN)
V[N][N] - მნიშვნელობების მატრიცა
lev - მომავალი ხაზის მნიშვნელობა („წყლის დონე“)
for (i გადის ყველა სტრიქონს 0-დან N-2-მდე)
  for (j გადის ყველა სვეტს 0-დან N-2-მდე)
    {
      ვიღებთ წერტილებს V[i][j], v[i][j+1], v[i+1][j+1], v[i+1][j],
      ვადარებთ მათ lev - ს, ვანგარიშობთ ამ
      ოთხი წვეროს განლაგების კოდს,
      და ვხაზავთ ამ კოდის შესაბამის ხაზებს მოცემულ კვადრატში
    }



ახლა კი დავუბრუნდეთ იმ შემთხვევებს (კოდით 5 და 10, „ოთხი წერტილის შემთხვევა“), სადაც წვეროების მნიშვნელობები ხაზის გავლების ერთმნიშვნელოვან სურათს ვერ ქმნიან:

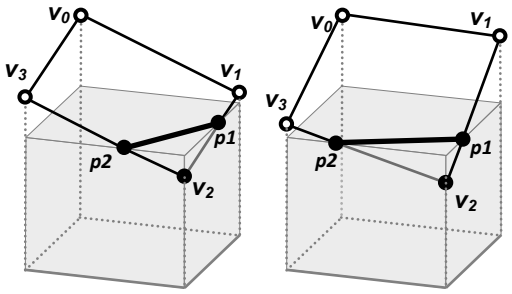
Saddle point problem:
მაგალითად, კვადრატისთვის კოდით 5, შესაძლებელია ორი ვარიანტი: ან მაღალ, (თეთრ) წერტილებს „ქედი“ აკავშირებს, ან პირიქით, დაბალი, შავი წერტილებია დაკავშირებული „ღრმულით“. ეს ორი ვარიანტი სხვადასხვანაირად აისახება ხაზებით.
ამ ორი ვარიანტიდან სწორის ასარჩევად ხშირად გამოიყენება შემდეგი წესი: იღება ოთხივე წვეროს გაშუალედებული სიმაღლე, და, თუ ეს სიმაღლე „წყლის დონეზე“ დაბალია, მაშინ ირჩევენ „ღრმულის“ ვერსიას, თუ კი საშუალო სიმაღლე „წყლის დონეზე“ მეტია, მაშინ - „ქედის“ ვერსიას.



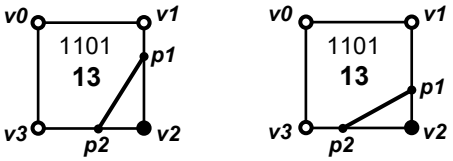
აქამდე ხაზის გასავლებად ვიყენებდით კვადრატის მხარეების შუა წერტილებს. ეს მარტივია და ზოგ შემთხვევაში საკმარისიც. ახლა კი განვიხილოთ უფრო ზუსტი მეთოდი.

ინტერპოლაციის მეთოდი.

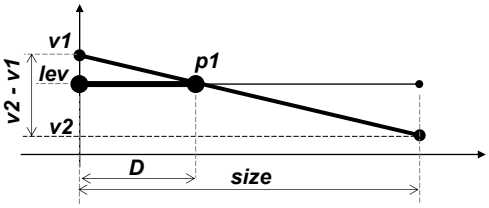
შუა წერტილების მეთოდთან შედარებით უფრო ხარისხიანი, მაგრამ ნელი მეთოდია **ინტერპოლაციის მეთოდი** - კვადრატის მხარის და „წყლის დონის" გადაკვეთის წერტილის დადგენა.



სურათზე ნაჩვენებია, რომ ერთი გა იგივე კოდის მქონე კვადრატში (მაგალითში - 1101) ხაზის გავლა კი ხდება ერთ და იგივე მხარეებს შორის, მაგრამ ამ ხაზის კონკრეტული კონფიგურაცია, ანუ კვადრატთან გადაკვეთის წერტილები სხვადასხვაა. ამ წერტილების კოორდინატები დამოკიდებულია კვადრატის შესაბამისი მხარის წვეროების სიმაღლეზე.

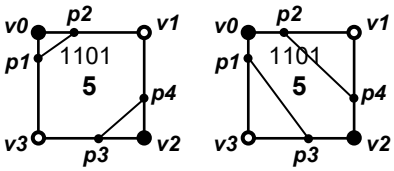


გამოვიყენოთ წრფივი ინტერპოლირება.
size - კვადრატის მხარის ზომა
v1, v2 - წვეროების სიმაღლე
D - დისტანცია პირველი წვეროდან გადაკვეთს წერტილამდე



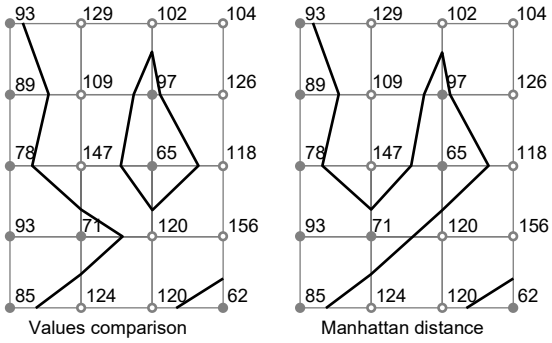
D = size*(lev-v1)/(v2-v1).
თუ ჩავთვლით, რომ კვადრატის მხარე (size) ერთის ტოლია, განტოლება კიდევ უფრო გამარტივდება.
აქვე გვახსოვდეს, რომ, გაყოფის გამო, უნდა გამოვიყენოთ float ტიპი.

ოთხი წერტილის პრობლემის (Saddle point problem) ამოხსნა ამ შემთხვევაში შეიძლება ასევე სხვა მეთოდით: ორ ვარიანტს შორის ავიღებთ ისეთს, რომელშიც ხაზებს ნაკლები ჯამური სიგრძე ექნებათ.



აქ პროცესის აჩქარების მიზნით, შეგვიძლია გამოვიყენოთ არა ეუკლიდეს, არამედ მანჰეტენის მეტრიკა, ანუ კვადრატების ჯამების გამოთვლის ნაცვლად მხოლოდ კოორდინატების სხვაობები ავიღოთ.

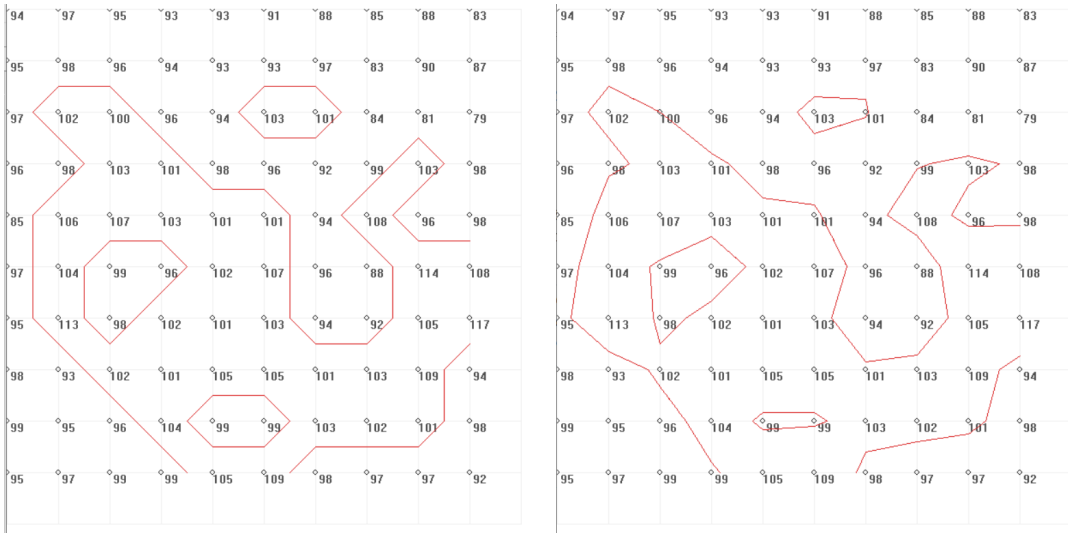
გადაწყვეტილების მიღების ორივე მეთოდი (წვეროების ჯამული შედარება წყლის დონესთან და მონაკვეთების ჯამური სიგრძეების შედარება) ევრისტიკულია და ზოგ შემთხვევაში გვაძლევს სხვადასხვა შედეგს. ჯამური სიგრძეების მეთოდის გამოყენებისას, მანჰეტენის და ეუკლიდეს მეტრიკების გამოყენება როგორც წესი, გვაძლევს იდენტურ შედეგს.



სურათზე კონკრეტულ მაგალითზე ნაჩვენებია ამ ორი მეთოდით მიღებული შედეგების განსხვავება. ორივე შემთხვევაში წყლის დონე ასის ტოლია.
პირველ შემთხვევაში აღებულია კვადრატის (147, 65, 120, 71) წვეროების გაშუალედებული სიმაღლე, რაც 403/4 - ასზე მაღალია, და მაშასადამე, წვეროებს 147 და 120 შორის გაიარა „ქედმა“.

წვეროები 97 და 65 ამ შემთხვევაში ქმნიან იზოლირებულ ღრმულს, ან „ტბას“.
მეორე შემთხვევაში კი უფრო მოკლე სანაპირო ხაზის პრიორიტეტმა მოგვცა ვარიანტი, რომელშიც წვეროები 97 და 65 - „ზღვის ყურეში“ მოხვდნენ, ხოლო წვეროები 109 და 147 „ნახევარკუნძულს“ ქმნიან.

შუა წერტილების და ინტერპოლაციის მეთოდების შედარება
სურათზე მოყვანილია შუა წერტილების და ინტერპოლაციის მეთოდებით გენერირებული კონტურები ერთი და იგივე მონაცემების მასივისთვის.



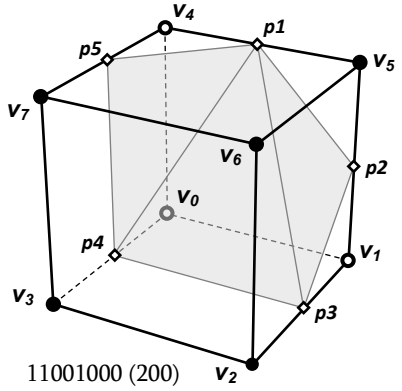
კუბების მარშის მეთოდი, marching cubes

მეთოდი არის კვადრატების მარშის მეთოდის ანალოგი სამგანზომილებიან სივრცეში.

სივრცეში მოცემულია რეგულარული ბადე ფუნქციის გარკეული მნიშვნელობებით. ასაგებია ზედაპირი, რომელიც გაჰყოფს სივრცეს ზონებად, სადაც მუნქციის მნიშვნელობა მოცემულ სიდიდეზე მეტია და სადაც ის ამ სიდიდეზე ნაკლებია.

კვადრატების მარშის მეთოდის მსგავსად აქაც იგება ბადეზე რეგულარული გეომეტრიული სტრუქტურა, ორონდ ამ შემთხვევაშ იუკვე კუბების. თითო კუბის ყველა წვეროსთვის მოცემულია მნიშვნელობა, კუბზე გადის ზედაპირი, რომელიც ჰყოფს ერთმანეთისგან წვეროებს (და კუბის სივრცეს) მოცემულზე დაბალი და მოცემუზლე მაღალი მნიშვნელობებით.

ისევე, როგორც კვადრატების მარშის მეთოდის დროს, გამოიყენება კოდირება. (Lookup table/index mapping?). ამ შემთხვევაში გაცილებით დიდია ელემენტობის მრაფალფეროვნება: კვადრატის 8 წვეროსთვის ორ-ორი შემთხვევა, ანუ $2^8 = 256$ ვარიანტი.



11001000 (200)

სურათზე ნაჩვენებია შემთხვევა, როდესაც წვეროების V_0,V_1 და V_4 მნიშვნელობები ზღურბლის მნიშვნელობაზე მაღლაა, ხოლო სხვა წვეროების - ნაკლებია. ამ შემთხვევას შეესაბამება კოდი 11001000 (ანუ 200) და კვეთის წვეროები წიბოებზე (V_4, V_5), (V_1, V_5), (V_1, V_2), (V_0, V_3), (V_4, V_7).

მაგალითში გამოყენებულია შუა წერტილების მეთოდი.

მიღებული წერტილები განსაზღვრავს სამ საკუთხეს: (P_1, P_4, P_5), (P_1, P_3, P_4) და (P_1, P_2, P_3).

ისევე, როგორც კვადრატების მარშისთვის, შესაძლებელია გადაკვეთის წერტილებისთვის როგორც შა წეეტილის, ასევე ინტერპოლაციის მეთოდების გამოყენება.

ასევე კუბების მარშისთვისაც არსებობს არაერთმნიშვნელოვანი ვარიანტები, რომლებისთვისაც გამოიყენება წვეროების გასაშუალებული მნიშვნელობის მეთოდი ან გადამკვეთი ზედაპირის ფართის ან კონტურის სიგრძის მინიმალური მნიშვნელობის მეთოდები.

თუ ამოცანა არ ითვალისწინებს მხოლოდ გარსის (მემბრანის) გენერირებას, არამეტ solid-გემომეტრიის აგებას (რაც უფრო ხშირი შემთხვევაა), მაშინ საჭიროა ასევე სამკუთხედების ნორმალების გამოთვლა და შენახვა შესაბამის მონაცემტა სტრუქტურაში.

მეთოდი ფართოდ გამოიყენება მონაცემების ვიზუალიზაციაში, ხელოვნური რელიეფების და ქვაბულების გერნერირებაში, კომპიუტერულ თამაშებში, კომპიუტერული გრაფიკის სხვა ამოცანებში.

Lookup table

In computer science, a **lookup table** (**LUT**) is an array that replaces runtime computation with a simpler array indexing operation, in a process termed as ***direct addressing***. The savings in processing time can be significant, because retrieving a value from memory is often faster than carrying out an "expensive" computation or input/output operation.[1] The tables may be precalculated and stored in static program storage, calculated (or "pre-fetched") as part of a program's initialization phase (*memoization*), or even stored in hardware in application-specific platforms. Lookup tables are also used extensively to validate input values by matching against a list of valid (or invalid) items in an array and, in some programming languages, may include pointer functions (or offsets to labels) to process the matching input. FPGAs also make extensive use of reconfigurable, hardware-implemented, lookup tables to provide programmable hardware functionality. LUTs differ …

In image processing, lookup tables are often called **LUT**s (or 3DLUT), and give an output value for each of a range of index values. One common LUT, called the *colormap* or *palette*, is used to determine the colors and intensity values with which a particular image will be displayed.

Index mapping (or **direct addressing**, or a **trivial hash function**) in computer science describes using an array, in which each position corresponds to a key in the universe of possible values.[1] The technique is most effective when the universe of keys is reasonably small, such that allocating an array with one position for every possible key is affordable. Its effectiveness comes from the fact that an arbitrary position in an array can be examined in constant time.

Index mapping

Avoid branching[edit]

Roger Sayle gives an example[2] of eliminating a multiway branch caused by a switch statement:

```
inline bool HasOnly30Days(int m)
{
	switch (m) {
	case 4:  // April
	case 6:  // June
	case 9:  // September
	case 11: // November
		return true;
	default:
		return false;
	}
}
```

Which can be replaced with a table lookup:

```
inline bool HasOnly30Days(int m)
{
	static const bool T[] = { 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0 };
	return T[m-1];
}
```